Bulutkent Studio — complete upload script (scripts/upload_video.py)

Full working script used by the channel owner to upload original videos to youtube.com/@bulutkenttv (OAuth 2.0, videos.insert with made-for-kids flag, thumbnails.set, playlistItems.insert, captions.insert, optional scheduled publishAt). Source repository: https://github.com/n8br6ny2h2-lab/youtube

Verified uploads made with this flow on 2026-11-06: QAgZAEXwukE, MW5BeOAoZmk, 96HH7GgYMH4, Eln4_H4KKS4 (private, made for kids, thumbnails set, added to playlists).

```
#!/usr/bin/env python3
"""Bulutkent Studio - complete YouTube upload script (used by the channel owner only).

Uploads one of the owner's own original videos to the owner's own channel through the
YouTube Data API v3 with OAuth 2.0, marks it as made for kids, sets the thumbnail,
adds it to a playlist, uploads Turkish captions and optionally schedules publication.

Usage:
  python scripts/upload_video.py --video renders/episode.mp4 --title "..." --description "..." \
      --thumbnail thumbnails/episode.png --playlist "Renkler ve Şekiller" --captions subtitles/episode.srt \
      [--publish-at 2026-11-10T15:00:00Z] [--privacy private|unlisted|public]

OAuth client: config/secrets/client_secret.json (Desktop app). Token: config/secrets/token.json (chmod 600).
Scopes: youtube, youtube.upload, youtube.force-ssl, yt-analytics.readonly
"""
import argparse
import os
import sys
import time

from google.auth.transport.requests import Request
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build
from googleapiclient.errors import HttpError
from googleapiclient.http import MediaFileUpload

SCOPES = ["https://www.googleapis.com/auth/youtube", "https://www.googleapis.com/auth/youtube.upload",
          "https://www.googleapis.com/auth/youtube.force-ssl", "https://www.googleapis.com/auth/yt-
analytics.readonly"]
SECRET = "config/secrets/client_secret.json"
TOKEN = "config/secrets/token.json"


def youtube():
    creds = Credentials.from_authorized_user_file(TOKEN, SCOPES) if os.path.exists(TOKEN) else None
    if creds and creds.expired and creds.refresh_token:
        creds.refresh(Request())
    if not creds or not creds.valid:
        creds = InstalledAppFlow.from_client_secrets_file(SECRET, SCOPES).run_local_server(port=8765)
    with open(TOKEN, "w") as fh:
        fh.write(creds.to_json())
    os.chmod(TOKEN, 0o600)
    return build("youtube", "v3", credentials=creds)


def upload(yt, args):
    status = {"privacyStatus": args.privacy, "selfDeclaredMadeForKids": True, "embeddable": True, "license":
"youtube"}
    if args.publish_at:
        status["privacyStatus"] = "private"
        status["publishAt"] = args.publish_at
    body = {"snippet": {"title": args.title, "description": args.description, "tags": args.tags.split(",") if
args.tags else [],
                        "categoryId": "1", "defaultLanguage": "tr", "defaultAudioLanguage": "tr"}, "status":
status}
    media = MediaFileUpload(args.video, chunksize=8 * 1024 * 1024, resumable=True, mimetype="video/mp4")
    request = yt.videos().insert(part="snippet,status", body=body, media_body=media)
    response, retry = None, 0
    while response is None:
        try:
            progress, response = request.next_chunk()
            if progress:
                print(f"uploaded {int(progress.progress() * 100)}%")
```

```
        except HttpError as e:
            if e.resp.status in (500, 502, 503, 504) and retry < 5:
                retry += 1
                time.sleep(2 ** retry)
                continue
            raise
    return response["id"]


def add_to_playlist(yt, title, video_id):
    pid = None
    for p in yt.playlists().list(part="snippet", mine=True, maxResults=50).execute().get("items", []):
        if p["snippet"]["title"] == title:
            pid = p["id"]
    if not pid:
        pid = yt.playlists().insert(part="snippet,status", body={"snippet": {"title": title, "defaultLanguage":
"tr"},
                                                                 "status": {"privacyStatus":
"public"}}).execute()["id"]
    yt.playlistItems().insert(part="snippet", body={"snippet": {"playlistId": pid,
                                                                "resourceId": {"kind": "youtube#video",
"videoId": video_id}}}).execute()


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--video", required=True)
    ap.add_argument("--title", required=True)
    ap.add_argument("--description", default="")
    ap.add_argument("--tags", default="")
    ap.add_argument("--thumbnail")
    ap.add_argument("--playlist")
    ap.add_argument("--captions")
    ap.add_argument("--privacy", default="private", choices=["private", "unlisted", "public"])
    ap.add_argument("--publish-at")
    args = ap.parse_args()
    yt = youtube()
    vid = upload(yt, args)
    print("video id:", vid, f"https://youtu.be/{vid}")
    if args.thumbnail:
        yt.thumbnails().set(videoId=vid, media_body=MediaFileUpload(args.thumbnail)).execute()
        print("thumbnail set")
    if args.playlist:
        add_to_playlist(yt, args.playlist, vid)
        print("added to playlist:", args.playlist)
    if args.captions:
        yt.captions().insert(part="snippet", body={"snippet": {"videoId": vid, "language": "tr", "name":
"Türkçe", "isDraft": False}},
                             media_body=MediaFileUpload(args.captions, mimetype="application/octet-
stream")).execute()
        print("captions uploaded")
    item = yt.videos().list(part="status,processingDetails", id=vid).execute()["items"][0]
    print("status:", item["status"]["privacyStatus"], "| made for kids:",
item["status"].get("selfDeclaredMadeForKids"),
          "| processing:", item.get("processingDetails", {}).get("processingStatus"))
    return 0


if __name__ == "__main__":
    sys.exit(main())
```